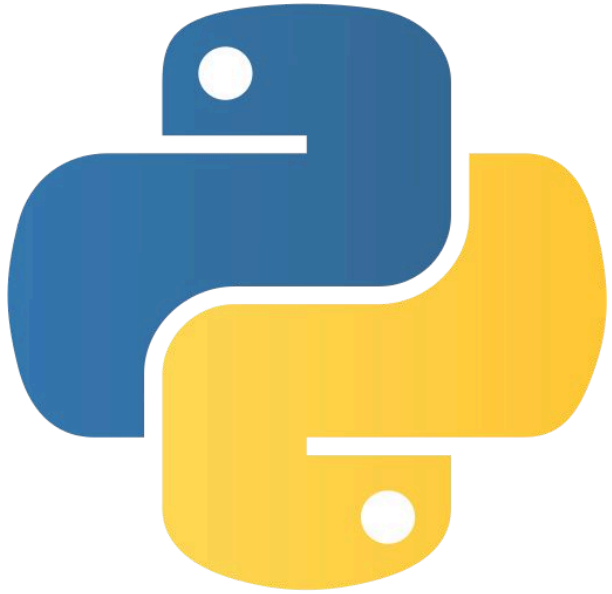
4 CLASE LIVE

Unit testing en Programación Orientada a Objetos

Módulo de Programación Avanzada



Álvaro Solano Jiménez

2026 - version 1.0

pontIA

Contenido

01 Unit testing + Integration testing

02 Ejercicio 6: añadir unit tests a nuestro código

01 ¿Qué es un unit test (test unitario)?

- Prueba automatizada que verifica el comportamiento de una unidad pequeña de código
- Esa unidad de código, normalmente suele ser una función, un método o una clase aislada
- El test unitario debe ser rápido, determinista y fácil de entender
- Idea clave: un test unitario comprueba una responsabilidad concreta del código

01 Características de un buen unit test

- Aislado: evita depender de base de datos, red, sistema de archivos u otros servicios
- Determinista: con las mismas entradas, produce siempre el mismo resultado
- Rápido: permite ejecutarlo muchas veces durante el desarrollo
- Repetible: funciona igual en local y en CI/CD
- Legible: el test debe explicar qué comportamiento se espera

01 Patrón AAA: Arrange -> Act -> Assert

- Arrange: prepara los datos, objetos y dependencias necesarias
- Act: ejecuta la operación que queremos comprobar
- Assert: verifica el resultado o efecto esperado
- Separar estas fases hace que los tests sean más claros y mantenibles

01 ¿Qué debemos probar?

Siempre debemos añadir tests que cubran la lógica o funcionalidades del código que hemos añadido nosotros. No hace falta añadir tests para comprobar que los métodos propios de Python, o de librerías externas funcionan.

- Casos normales: hay que testear el comportamiento esperado de nuestro código, con entradas válidas y valores corrientes. El 99% de las veces el código irá por este camino
- Casos límite: valores mínimos, máximos, vacíos o inesperados. Hay que asegurarse de que cualquier valor, por muy absurdo que parezca, que llegue a nuestro código es gestionado de una forma correcta
- Errores: excepciones y validaciones
- Reglas de negocio: condiciones y decisiones relevantes

01 Unit Testing vs. Integration Testing

- Unit Testing: prueba una unidad de código de forma aislada
- Integration Testing: comprueba que varias piezas funcionan correctamente juntas
- Ejemplo: servicio + repositorio + base de datos real o de prueba
- Los tests de integración suelen ser más lentos y requieren más infraestructura
- Ambos se complementan: unit tests para feedback rápido y tests de integración para validar interacciones reales

02 Ejercicio 6: añadir unit tests a nuestra aplicación de ajedrez online

En este ejercicio la idea es trabajar:

- Qué son los unit tests y de qué partes se componen
- Cómo añadir unit tests al código existente
- El concepto de “mock / mocking” y aplicarlo a un test



02 Ejercicio 6: añadir unit tests a nuestra aplicación de ajedrez online

¡Vamos al código!

Abrid el enunciado: `“poo_ejercicio_6_enunciado.md”`



pontIA

Muchas gracias por vuestra atención

Álvaro Solano Jiménez

www.pontia.tech

alvaro.solano@centricsoftware.com